

LOYOLA-ICAM COLLEGE OF ENGINEERING AND TECHNOLOGY
(Autonomous)
Loyola Campus, Nungambakkam, Chennai – 600 034

INVENTORY MANAGEMENT SYSTEM

A MINI PROJECT REPORT

Submitted by

MUTHEEUR RAHMAN C
(Register No: 311124205036)

BACHELOR OF TECHNOLOGY

IN

INFORMATION TECHNOLOGY

LOYOLA-ICAM COLLEGE OF ENGINEERING AND TECHNOLOGY
ANNA UNIVERSITY: CHENNAI 600 025

Apr 2026

BONAFIDE CERTIFICATE

Certified that this project report titled **"E-MART INVENTORY MANAGEMENT SYSTEM"** is the bonafide work of **MUTHEEUR RAHMAN C (311124205036)**, who carried out the project work under my supervision.

SIGNATURE
Dr G Shobana
Project Coordinator
Department of Information Technology
Loyola-ICAM College of Engineering and Technology
Loyola Campus,
Nungambakkam, Chennai-34

SIGNATURE
Ms. Sherril Sophie Maria Vincent
Head of the Department
Department of Information Technology
Loyola-ICAM College of Engineering and Technology
Loyola Campus,
Nungambakkam, Chennai-34

ACKNOWLEDGEMENT

First of all, we thank The Lord Almighty for showering His blessings on us, which enabled us to complete this project successfully.

We express our sincere gratitude to our beloved and respected **Director, Rev. Dr. Justine SJ.** We wish to acknowledge with thanks to our college, **Dean of Students, Rev. Dr. Albert William SJ** for his support and encouragement.

We extend our sincere gratitude to **Dr. A. Baskar** , **Principal**, for his motivation and support towards this project.

We express our sincere gratitude to the **Head of the Department** of Information Technology, **Ms Sherril Sophie Maria Vincent** for her constant support, motivation, encouragement and being a source of inspiration throughout this project and during the course of this semester.

Our sincere thanks to our **Project Coordinator Dr. G Shobana** for their valuable guidance in shaping this project.

We also take this opportunity to thank all the **Faculty and Non-teaching staff members** of Department of Information Technology for their constant support. Finally, we thank each and every one who helped us to complete this project.

MUTHEEUR RAHMAN C

ABSTRACT

Managing inventory efficiently is one of the core challenges faced by retail and e-commerce businesses today. Manual tracking using spreadsheets or paper-based methods is error-prone, time-consuming and fails to scale as the number of products grows. The E-Mart Inventory Management System is a web-based application developed to address these issues by providing a centralised, database-driven solution for tracking and managing product inventory in a small-to-medium scale retail setup.

This project is developed as part of the Database Management System (CS24302) course to demonstrate practical application of database concepts including schema design, CRUD operations, aggregation pipelines and audit logging. The system uses MongoDB as the primary database, with Node.js and Express.js forming the backend server layer, and React.js with Tailwind CSS used for building the frontend interface.

The system provides four key modules — a live Dashboard, an Inventory management panel, an Audit Log viewer, and a Smart Expiry Predictor. The Dashboard displays real-time statistics such as total products, inventory value, low stock count and expiry alerts, powered by MongoDB aggregation pipelines. The Inventory module allows users to add and delete products, search and filter records, and view product status badges. Every add or delete action is automatically captured in the Audit Log collection, maintaining a complete trail of inventory changes. The Smart Expiry Predictor module calculates risk scores for products based on their stock quantity, daily sales rate and remaining shelf life, giving the store manager actionable recommendations such as applying discounts or monitoring closely.

The application follows a RESTful API architecture and is built with a clean separation between the frontend and backend. All database interactions are handled through Mongoose schemas with proper validation, and the user interface provides a real-time connection indicator showing MongoDB status. This project demonstrates how modern full-stack web technologies can be used to solve real-world inventory management problems using core database management principles.

TABLE OF CONTENTS

BONAFIDE CERTIFICATE i
ACKNOWLEDGEMENT ii
ABSTRACT iii
TABLE OF CONTENTS iv
LIST OF ABBREVIATIONS v

CHAPTER 1 – INTRODUCTION 1
1.1 Problem Statement 1
1.2 Overview of the Project 2
1.3 Purpose and Scope 3

CHAPTER 2 – REQUIREMENTS 4
2.1 Functional Requirements 4
2.2 Non-Functional Requirements 5

CHAPTER 3 – ANALYSIS 6
3.1 Bull Diagram 6
3.2 Octopus Diagram 7

CHAPTER 4 – SYSTEM DESIGN 8
4.1 System Architecture 8
4.2 Data Flow Diagram 9
4.3 Database Schema Design 10

CHAPTER 5 – MODULE DESCRIPTION 11
5.1 Dashboard Module 11
5.2 Inventory Module 12
5.3 Audit Log Module 13
5.4 Smart Expiry Predictor Module 14

CHAPTER 6 – CONCLUSION 15
6.1 Conclusion 15
6.2 Future Enhancements 15

CHAPTER 7 – REFERENCES 16

CHAPTER 8 – APPENDIX: SAMPLE CODE 17

LIST OF ABBREVIATIONS

ABBREVIATION	FULL FORM
DBMS	Database Management System
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
REST	Representational State Transfer
JSON	JavaScript Object Notation
UI	User Interface
SKU	Stock Keeping Unit
INR	Indian National Rupee
MVC	Model View Controller
HTTP	HyperText Transfer Protocol
CORS	Cross Origin Resource Sharing
DFD	Data Flow Diagram
SRS	Software Requirements Specification
IDE	Integrated Development Environment
CSS	Cascading Style Sheets
JSX	JavaScript XML
NPM	Node Package Manager
ODM	Object Document Mapper

CHAPTER 1

INTRODUCTION

1.1 Problem Statement

Retail businesses, particularly small grocery stores and supermarkets, deal with a large number of products every day. Keeping track of product quantities, prices, supplier details, and expiry dates manually is a difficult and often inaccurate process. Without a proper system in place, businesses frequently face problems such as overstocking, understocking, selling expired products, and inability to track inventory changes over time.

The E-Mart Inventory Management System is developed to solve these practical problems by providing a single, easy-to-use web application where store managers can add products, monitor stock levels, check expiry dates, and view a complete history of inventory actions. The system makes use of a MongoDB database to store and retrieve data efficiently, and provides real-time statistics to support better decision-making.

1.2 Overview of the Project

The E-Mart Inventory Management System is a full-stack web application built using the MERN-like technology stack — MongoDB for the database, Express.js and Node.js for the backend server, and React.js with Tailwind CSS for the frontend interface. The system follows a RESTful API architecture where the frontend communicates with the backend through well-defined HTTP endpoints.

The project is primarily focused on demonstrating practical usage of database management concepts such as schema design, CRUD operations, aggregation pipelines, indexing and audit logging. Each product in the system is stored with fields including product name, SKU (Stock Keeping Unit), price, quantity, category, supplier name and expiry date. The system automatically calculates virtual fields such as stock status and expiry status based on the stored data.

The application has four main functional areas: a real-time dashboard that shows key statistics, an inventory management panel for adding and viewing products, an audit log that records every

change made, and a smart expiry predictor that assesses the risk of products spoiling before they can be sold.

1.3 Purpose and Scope

The main purpose of this project is to develop a working prototype of an inventory management system that can be used in a small retail setup such as a grocery store or a mini supermarket. The system is designed to be simple to use, accurate in data management and useful for day-to-day store operations.

The scope of the project includes the following:

- Adding new products to the inventory with all required details such as name, SKU, price, quantity, category, supplier and expiry date.
- Viewing and searching the product list with options to filter by category or stock status.
- Deleting products that are no longer available or have expired.
- Viewing a dashboard with real-time statistics such as total products, total inventory value, low stock alerts and expiry warnings.
- Maintaining a complete audit trail of all add and delete actions performed on the inventory.
- Predicting expiry risk for products based on their current stock levels and estimated daily sales rate.

The system does not currently cover features like user authentication, purchase order generation, supplier management, or barcode scanning. These can be added in future versions of the system.

CHAPTER 2

REQUIREMENTS

2.1 Functional Requirements

The following are the functional requirements of the E-Mart Inventory Management System:

1. The system shall allow the user to add a new product to the inventory by providing details such as product name, SKU, price, quantity, category, supplier name and expiry date.
2. The system shall display a complete list of all products currently stored in the database, sorted by the date they were added.
3. The system shall allow the user to delete a product from the inventory using its unique MongoDB document ID.
4. The system shall generate and display an audit log entry automatically whenever a product is added or deleted, capturing the full product snapshot and a human-readable description of the action.
5. The system shall display a dashboard showing real-time statistics including total number of products, total units in stock, total inventory value in rupees, low stock count and out-of-stock count.
6. The system shall identify and display products that are expiring within the next seven days and within the next thirty days.
7. The system shall allow the user to search for products by name or SKU from the inventory listing page.
8. The system shall allow the user to filter the product list by category or stock status.
9. The system shall display a category-wise breakdown of products on the dashboard.
10. The system shall calculate and display a risk score for each product in the Smart Expiry Predictor, based on the quantity in stock, daily sales rate and the number of days until expiry.
11. The system shall display the most recent audit log entries on the dashboard, providing a quick view of recent inventory activity.
12. The system shall validate all product input fields before saving to the database and display appropriate error messages for missing or invalid data.
13. The system shall show a real-time database connection status indicator in the sidebar, updating automatically based on the backend health check response.
14. The system shall prevent duplicate product entries by enforcing a unique constraint on the SKU field.
15. The system shall calculate and display virtual status badges for each product — such as In Stock, Low Stock, Out of Stock, Expiring Soon and Expired — based on current quantity and expiry date values.

2.2 Non-Functional Requirements

The following are the non-functional requirements of the system:

- Performance: The system should load dashboard statistics and inventory data within two to three seconds under normal network conditions.
- Reliability: The MongoDB database connection should be monitored continuously, and the application should display a disconnected status if the connection is lost.
- Usability: The user interface should be clean, intuitive and easy to navigate, even for users with limited technical experience.
- Maintainability: The codebase should be modular and well-commented, with a clear separation between frontend components, backend controllers, models and routes.
- Scalability: The database schema and API design should be capable of supporting a larger product catalog without requiring major architectural changes.
- Compatibility: The frontend application should render correctly on modern web browsers including Google Chrome, Mozilla Firefox and Microsoft Edge.
- Security: All input data should be validated on the server side before being stored in the database to prevent invalid or malformed entries.

CHAPTER 3

ANALYSIS

3.1 Bull Diagram

The Bull Diagram (also known as the Context Diagram or Black Box Diagram) represents the system from an external perspective, showing what goes into the system and what comes out, without revealing the internal workings. It helps define the boundary of the system and identify its primary inputs, outputs and external actors.

For the E-Mart Inventory Management System, the Bull Diagram is described as follows:

Inputs to the System:

- Product details provided by the store manager: name, SKU, price, quantity, category, supplier and expiry date.
- User actions such as adding a product, deleting a product, or searching and filtering the inventory.
- Navigation requests to different pages: Dashboard, Inventory, Audit Log, Smart Expiry Predictor.

The System (Black Box):

The E-Mart Inventory Management System processes user inputs, performs database operations (insert, read, delete, aggregate), and returns the results to the user interface.

Outputs from the System:

- A list of all products stored in the inventory database.
- Dashboard statistics such as total products, inventory value, low stock count and expiry alerts.
- An audit log showing all past add and delete actions.
- Risk scores and recommendations from the Smart Expiry Predictor.
- Status badges and alerts for products with low stock or upcoming expiry.

External Actors:

- Store Manager / User — the primary actor who interacts with the system through the web interface.
- MongoDB Database — the external data store that persists all product and audit log records.

3.2 Octopus Diagram

The Octopus Diagram (also called the Stakeholder Map or Spider Diagram) extends the Bull Diagram by showing all external entities or stakeholders that interact with the system, along with the nature of their interaction. Each 'tentacle' of the octopus represents a different entity connected to the system.

The following entities interact with the E-Mart Inventory Management System:

16. Store Manager — The primary user of the system. Adds products, deletes expired items, views inventory, checks the dashboard and reviews audit logs.
17. MongoDB Database Server — Stores and retrieves all product records, audit log entries and aggregation query results.
18. Express.js Backend Server — Acts as the middleware that processes API requests from the frontend and communicates with the MongoDB database.
19. React Frontend Application — Provides the web-based user interface through which the store manager interacts with the system.
20. Node.js Runtime Environment — Executes the backend server code and handles all asynchronous operations.
21. Product Suppliers (Indirect) — Supplier names are stored in the product records, linking products to their respective vendors for reference.

The Octopus Diagram shows that the Store Manager is the central actor, connected to the system through the frontend, which in turn communicates with the backend server, which connects to the MongoDB database. All information flows back through this chain to the store manager.

CHAPTER 4

SYSTEM DESIGN

4.1 System Architecture

The E-Mart Inventory Management System follows a three-tier client-server architecture. The three tiers are the presentation tier (frontend), the application/business logic tier (backend), and the data tier (database).

Presentation Tier (Frontend):

The frontend is built using React.js and styled with Tailwind CSS. It consists of multiple pages — Dashboard, Inventory, Audit Log, and Smart Expiry Predictor — with a shared Sidebar for navigation. The frontend communicates with the backend via Axios HTTP requests to the RESTful API endpoints.

Application Tier (Backend):

The backend is a Node.js and Express.js application that exposes several API endpoints. It processes incoming HTTP requests, applies business logic such as validation and risk score calculation, and interacts with the MongoDB database through Mongoose ODM. The backend runs on port 5000 by default.

Data Tier (Database):

MongoDB is used as the database, running locally on port 27017. The database is named 'emart_inventory' and contains two main collections: 'products' and 'auditlogs'. Mongoose schemas enforce the data structure and validation rules for both collections.

4.2 Data Flow Diagram

The Data Flow Diagram (DFD) describes how data moves through the system from the user to the database and back.

Level 0 DFD (Context Diagram):

At the highest level, the store manager sends inputs such as product data and action requests to the E-Mart system. The system returns outputs such as product listings, statistics, audit logs and risk reports.

Level 1 DFD:

At the next level, the system is broken into four main processes:

22. Product Management Process — Receives product data from the user and stores it in the Products collection. Also reads from the Products collection to return the inventory list.
23. Statistics Generation Process — Reads all product records from the Products collection and applies MongoDB aggregation pipelines to compute dashboard statistics, which are then displayed to the user.
24. Audit Logging Process — Whenever a product is added or deleted, this process writes a new audit log record to the AuditLogs collection with a full snapshot of the product and a description of the action.
25. Expiry Risk Assessment Process — Reads all product records, computes a risk score based on quantity, daily sales rate and days to expiry, and returns risk levels and recommendations to the user.

4.3 Database Schema Design

The database has two collections with the following schema definitions:

Products Collection Schema:

- name (String, required) — The full name of the product.
- sku (String, required, unique) — Stock Keeping Unit identifier stored in uppercase.
- price (Number, required) — The selling price of the product in Indian Rupees.
- quantity (Number, required) — Number of units currently in stock.
- category (String, enum) — Product category selected from a fixed list of ten options including Dairy, Bakery, Produce, Frozen, Beverages, Snacks, Meat, Personal Care, Household, and Other.
- supplier (String, required) — Name of the product supplier or vendor.
- expiryDate (Date, required) — The date after which the product should not be sold.
- dailySalesRate (Number, optional) — Average units sold per day, used by the expiry risk predictor.
- createdAt, updatedAt (Date, auto) — Automatically maintained timestamps by Mongoose.

AuditLogs Collection Schema:

- action (String) — The type of action performed, either ADD_PRODUCT or DELETE_PRODUCT.

- productSnapshot (Mixed) — A complete copy of the product document at the time of the action.
- description (String) — A human-readable message describing what happened.
- performedAt (Date) — The timestamp when the action was recorded.

CHAPTER 5

MODULE DESCRIPTION

5.1 Dashboard Module

The Dashboard is the home page of the application and gives the store manager a quick overview of the current state of the inventory. It is implemented in the Dashboard.jsx component on the frontend and is powered by the /stats and /api/expiry-risk API endpoints on the backend.

The dashboard displays the following information:

- Total number of distinct products currently in the inventory.
- Total units across all products combined.
- Total inventory value calculated as the sum of (price x quantity) for all products.
- Number of products with low stock (between 1 and 10 units remaining).
- Number of products that are completely out of stock.
- Number of products that have already expired.
- Number of products expiring within the next 7 days and within 30 days.
- A category-wise breakdown showing how many products belong to each category.
- A list of recent audit log entries showing the latest add and delete actions.
- A quick expiry risk list showing products that are approaching their expiry date soon.

The dashboard statistics are fetched using a MongoDB aggregation pipeline that runs multiple queries in parallel using Promise.all(), ensuring fast response times. Each statistic is displayed as a card with an icon and colour coding for quick visual interpretation.

5.2 Inventory Module

The Inventory module is the main operational module of the system. It is implemented in the Inventory.jsx page and allows the store manager to view, search, filter, add and delete products. The module communicates with the /products API endpoints on the backend.

Key features of the Inventory module include:

- A full table listing of all products sorted by the date they were added, with columns for name, SKU, category, supplier, price, quantity, expiry date and status badges.
- A search bar that filters the product list in real time based on the product name or SKU.
- Category and stock status filter dropdowns to narrow down the displayed products.

- An Add Product button that opens a modal form where the manager can enter all required product details and submit the form to add the product to the database.
- A Delete button for each product row that removes the product from the inventory and logs the action to the audit log.
- Visual status badges for each product — In Stock (green), Low Stock (yellow), Out of Stock (red), Expiring Soon (orange), and Expired (red) — determined dynamically from the product data.

The Add Product modal includes client-side validation to ensure all required fields are filled before the form is submitted. Server-side validation is also applied in the backend controller, and duplicate SKU entries are rejected with an appropriate error message.

5.3 Audit Log Module

The Audit Log module provides a complete history of all add and delete operations performed on the inventory. It is implemented in the AuditLog.jsx page and fetches data from the /audit endpoint, which returns the last 50 audit log entries sorted by the most recent action first.

Each audit log entry displays:

- The action type — ADD or DELETE — shown with a colour-coded badge.
- A human-readable description of the action, for example: 'Product Amul Milk (SKU: DAIRY-001) added to inventory. Qty: 120, Price: Rs. 62'.
- The date and time when the action was performed.
- A snapshot of the product at the time of the action.

Audit logging is handled entirely on the backend. Whenever a product is added or deleted, the product controller automatically creates a new AuditLog document using the AuditLog Mongoose model. This means the audit log cannot be manually altered from the frontend, ensuring data integrity.

5.4 Smart Expiry Predictor Module

The Smart Expiry Predictor is an intelligent module that calculates a risk score for each product based on how likely it is that the product will expire before it can be sold. It is implemented in the SmartExpiryPredictor.jsx page and uses the /api/expiry-risk endpoint on the backend.

The risk score is computed using the following logic:

- Days to Sell = Quantity in Stock / Daily Sales Rate
- Risk Score = (Days to Sell / Days Until Expiry) x 100

If the daily sales rate is not stored for a product, the system uses category-based default values — for example, Dairy products default to 8 units per day, Snacks to 10 units per day, and Household items to 1 unit per day.

Based on the calculated risk score, each product is assigned a risk level:

- Safe (score below 40) — No action needed. The product is expected to be sold well before its expiry date.
- Watch (score 40 to 79) — The product should be monitored. Sales are slower relative to the time remaining.
- At Risk (score 80 and above) — Immediate action is recommended. Applying a discount is suggested to increase the rate of sale and prevent the product from expiring.

The module displays a summary at the top showing the total number of at-risk products, the average risk score, and the highest-risk product. All products are listed in a table sorted from highest risk to lowest, allowing the store manager to quickly identify which products need attention first.

CHAPTER 6

CONCLUSION

6.1 Conclusion

The E-Mart Inventory Management System was successfully designed and implemented as a full-stack web application using MongoDB, Express.js, React.js and Node.js. The project demonstrates a practical application of database management concepts including relational schema design using Mongoose, CRUD operations through RESTful APIs, aggregation pipelines for real-time statistics, and audit logging for data accountability.

Through the development of this project, the following outcomes were achieved:

- A working product management system that allows adding, viewing and deleting inventory items with proper data validation.
- A real-time dashboard powered by MongoDB aggregation queries providing key business metrics.
- An automated audit trail that captures every inventory change without any manual effort from the user.
- A smart risk assessment feature that helps identify products that are likely to expire before being sold, and provides actionable recommendations.

The system proved that even a relatively simple database-driven web application can provide significant practical value when properly designed. The use of MongoDB's aggregation framework, combined with Mongoose's schema validation, provided a robust and reliable data layer for the application.

6.2 Future Enhancements

The following enhancements can be made to the system in future versions:

- User Authentication and Role-Based Access Control — Adding a login system with different roles such as Admin and Staff, so that only authorised users can perform sensitive operations like deleting products.
- Barcode Scanner Integration — Allowing products to be scanned and added directly using a barcode scanner, reducing manual data entry errors.
- Purchase Order Management — Adding the ability to generate purchase orders when stock falls below a defined reorder level.
- Sales Tracking — Recording sales transactions and using actual sales data to improve the accuracy of the Smart Expiry Predictor.

- Export and Reporting — Providing options to export inventory reports and audit logs as PDF or CSV files.
- Email Alerts — Sending automatic email notifications to the store manager when products are low in stock or approaching expiry.
- Supplier Management Module — Maintaining a dedicated database of suppliers with contact information linked to product records.

CHAPTER 7

REFERENCES

The following books, documentation and online resources were referred to during the development of this project:

26. Ramez Elmasri and Shamkant B. Navathe, "Fundamentals of Database Systems", 7th Edition, Pearson Education, 2016.
27. Abraham Silberschatz, Henry F. Korth and S. Sudarshan, "Database System Concepts", 7th Edition, McGraw-Hill Education, 2020.
28. MongoDB Official Documentation — "MongoDB Manual", Available at: https://www.mongodb.com/docs/manual/ [Accessed April 2025].
29. Mongoose Official Documentation — "Mongoose v8 Documentation", Available at: https://mongoosejs.com/docs/ [Accessed April 2025].
30. Express.js Official Documentation — "Express 4.x API Reference", Available at: https://expressjs.com/en/4x/api.html [Accessed April 2025].
31. React Official Documentation — "React — A JavaScript Library for Building User Interfaces", Available at: https://react.dev/ [Accessed April 2025].
32. Tailwind CSS Official Documentation — "Tailwind CSS v3 Documentation", Available at: https://tailwindcss.com/docs [Accessed April 2025].
33. Node.js Official Documentation — "Node.js v18 Documentation", Available at: https://nodejs.org/en/docs [Accessed April 2025].
34. Axios Documentation — "Axios HTTP Client", Available at: https://axios-http.com/docs/intro [Accessed April 2025].
35. Vite Official Documentation — "Vite Next Generation Frontend Tooling", Available at: https://vitejs.dev/guide/ [Accessed April 2025].

CHAPTER 8

APPENDIX — SAMPLE CODE

8.1 Product Schema (Product.js)

The following is the Mongoose schema definition for the Product collection stored in MongoDB. This schema enforces data types, required fields and validation rules for each product document.

```
const mongoose = require('mongoose');

const productSchema = new mongoose.Schema({
  name:        { type: String, required: true, trim: true },
  sku:         { type: String, required: true, unique: true, uppercase: true },
  price:       { type: Number, required: true, min: 0 },
  quantity:    { type: Number, required: true, min: 0 },
  category:    { type: String, enum: ['Dairy','Bakery','Produce',
                 'Frozen','Beverages','Snacks','Meat',
                 'Personal Care','Household','Other'], default:'Other' },
  supplier:    { type: String, required: true, trim: true },
  expiryDate:  { type: Date, required: true },
  dailySalesRate: { type: Number, default: null }
}, { timestamps: true, versionKey: false });

module.exports = mongoose.model('Product', productSchema);
```

8.2 Add Product API Endpoint (productController.js)

The following controller function handles POST requests to the /products/add endpoint. It validates the input, creates a new product document in MongoDB, and writes an audit log entry automatically.

```
const addProduct = async (req, res) => {
  try {
    const { name, sku, price, quantity,
            category, supplier, expiryDate } = req.body;
    if (!name || !sku || price==null || quantity==null
        || !supplier || !expiryDate) {
      return res.status(400).json({ success: false,
        message: 'Missing required fields' });
    }
    const product = await Product.create({
      name, sku, price, quantity,
      category: category || 'Other', supplier, expiryDate
    });
    await AuditLog.create({
      action: 'ADD_PRODUCT',
      productSnapshot: product.toObject(),
      description: `Product '${product.name}' added.`
    });
    res.status(201).json({ success: true, data: product });
  } catch (err) {
    if (err.code === 11000) return res.status(409).json(
      { success: false, message: 'SKU already exists' });
    res.status(500).json({ success: false, message: err.message });
  }
};
```

8.3 Dashboard Stats Aggregation (statsController.js)

The following code demonstrates the use of MongoDB aggregation pipelines to compute real-time dashboard statistics. Multiple queries are executed in parallel using Promise.all() for efficiency.

```
const getStats = async (_req, res) => {
  try {
    const now      = new Date();
    const in7Days  = new Date(now.getTime()
                    + 7 * 24 * 60 * 60 * 1000);
    const [ totalProducts, lowStockCount,
            outOfStockCount, expiringSoonCount,
            categoryBreakdown ] = await Promise.all([
      Product.countDocuments(),
      Product.countDocuments({ quantity: { $gt:0, $lte:10 } }),
      Product.countDocuments({ quantity: 0 }),
      Product.countDocuments({
        expiryDate: { $gte: now, $lte: in7Days }
      }),
      Product.aggregate([
        { $group: { _id: '$category',
          count: { $sum: 1 },
          totalQty: { $sum: '$quantity' } } },
        { $sort: { count: -1 } }
      ]),
    ]);
    res.json({ success: true, data: {
      totalProducts, lowStockCount,
      outOfStockCount, expiringSoonCount,
      categoryBreakdown } });
  } catch(err) {
    res.status(500).json({ success:false,
      message: err.message });
  }
};
```

8.4 Expiry Risk Score Calculation

The following logic is used in the expiryRiskController.js to compute risk scores for each product. Products with a risk score of 80 or above are marked as 'At Risk' and recommended for discounting.

```
const daysToSell = qty / dailySalesRate;
const daysUntilExpiry = (new Date(p.expiryDate) - now)
                        / (1000 * 60 * 60 * 24);
const riskScore = daysUntilExpiry <= 0 ? 999 :
  Math.round((daysToSell / daysUntilExpiry) * 100);

let riskLevel, recommendation;
if (riskScore >= 80) {
  riskLevel = 'At Risk';
  recommendation = 'Apply discount';
} else if (riskScore >= 40) {
  riskLevel = 'Watch';
  recommendation = 'Monitor';
} else {
  riskLevel = 'Safe';
  recommendation = 'No action needed';
}
```